DUAL V2.2 / ROBOT SOUL / ARCHITECTURE REVIEW

Agent + Skill 架构梳理

Agent + Skill Architecture Review

从“ VLM 直接调用函数 ”到“ Plan → Approval → Task → Executor ”的受监督机器人运行时。

From direct VLM function calling to a supervised robot runtime built around Plan → Approval → Task → Executor.

当前实现 / Current implementation: dual_v2_2_agent

审查日期 / Review date: 2026-08-27

EXECUTIVE SUMMARY

核心判断 / Executive Assessment

中文

新架构的本质不是“ 换了一个模型 ”。它重新定义了模型、网页、Agent、Skill、Tool 与底层控制之间的权限边界。

传统 VLM + Functions 模式让模型选择并调用函数，规划和执行容易发生在同一链路中。当前实现让模型或规则 Planner 只产生高层语义 Plan；Plan 本身不运动机器人，必须经过白名单校验、现场检查和逐 Plan 批准，才会创建 Hardware Task。

Python Agent 是机械臂、相机与感知模型的唯一软件所有者。Skill 负责完整任务策略，Tool 负责原子能力，RobotRAM 继续承担已验证的感知、规划与执行。

最大收益：安全权威分离、任务可观测、能力可复用、故障可恢复。

主要代价：进程和状态更多、策略存在双份校验、Planner 泛化能力仍有限。

ENGLISH

The new architecture is not merely a model replacement. It reallocates authority across the model, UI, Agent, Skills, Tools, and control stack.

In a conventional VLM + Functions design, the model chooses functions and planning may flow directly into dispatch. The current implementation allows a rule-based or LLM Planner to propose only a high-level semantic Plan. A Plan is inert until allowlist validation, on-site checks, and per-Plan approval create a Hardware Task.

The Python Agent is the sole software owner of arms, cameras, and perception models. Skills own task strategy, Tools expose atomic capabilities, and RobotRAM continues to provide the validated perception, planning, and execution stack.

Primary gains: separated safety authority, observability, reuse, and recovery.

Main costs: more processes and state, duplicated policy checks, and limited planner generality.

指标 / Metric	当前实现 / Current implementation
高层能力	5 个内建 Skills；其中 4 个进入网页/Harness 真机执行白名单。
原子能力	3 个感知 Tools + 5 个操作 Tools。
硬件所有权	1 个 Python Agent 持有机械臂、相机与感知模型。
启动模式	start：直接真机；console：网页在线、硬件 offline，现场一键启动。

LAYERED ARCHITECTURE

层次结构 / System Layers

层 / Layer	实现 / Implementation	职责 / Responsibility
01 操作入口	Robot Soul GUI + Harness	目标输入、Plan 审查、现场审批、Task Timeline。 Interfaces for goals, review, approval, and status.
02 策略与审批	TypeScript + Python gates	Skill 白名单、参数、动作预算、Plan TTL、checklist 与确认口令。
03 Agent Server	robot_agent/server.py	Plan/Task、硬件锁、异步执行、事件、取消与审批请求。
04 Planner	robot_agent/planner.py	规则优先、可选 LLM；自然语言 → SkillCall Plan。
05 Executor	robot_agent/executor.py	依赖校验、执行时语义解析、逐 Skill 执行与事件发送。
06 Skill	robot_agent/skills/	完整任务策略：感知、抓放、整理、恢复和启动。
07 Tool	robot_agent/tools/	3 个感知 + 5 个操作元动作；统一返回 ToolResult。
08 身体执行层	RobotRAM + CuRobo/Upper/RT	Step1 – 5、轨迹规划、UDP 控制与物理硬件。

中文职责边界

- Plan：纯数据，包含 instruction、SkillCall 与 constraints。
- Skill：任务级组合，封装顺序、前置条件、失败恢复与业务语义。
- Tool：确定性原子能力，是进入 RobotRAM 的受控适配层。
- RobotContext：持有相机、机械臂、模型、标定和当前场景状态。
- RobotRAM：底层能力被复用，而非被重新实现。

ENGLISH RESPONSIBILITIES

- Plan: inert data containing instruction, SkillCalls, and constraints.
- Skill: task composition owning order, preconditions, recovery, and semantics.
- Tool: deterministic atomic capability and controlled RobotRAM adapter.
- RobotContext: owns arms, cameras, models, calibration, and scene state.
- RobotRAM: existing low-level capabilities are reused, not rewritten.

关键不变量：模型不能提交关节角、CAN、shell、任意坐标或任意原子调用。

Key invariant: the model cannot submit joints, CAN, shell, arbitrary poses, or arbitrary atomic calls.

EXECUTION FLOW

真实执行链路 / End-to-End Execution

阶段 / Stage	中文说明	English
1 Goal	用户输入“ 整理桌面 ”。	The operator submits “ Clean the table ”.
2 Plan	Planner 生成 tidy_table SkillCall；不初始化硬件。	The Planner emits a tidy_table SkillCall without hardware bootstrap.
3 Approval	校验白名单、TTL、现场 checklist 与确认口令。	Validate allowlist, TTL, on-site checklist, and confirmation phrase.
4 Task	Server 创建唯一活动 Task，并获取 hardware lock。	The Server creates the sole active Task and acquires the hardware lock.
5 Perception	Ground → Estimate → Localize，建立当前场景。	Ground → Estimate → Localize builds the current scene.
6 Skill	tidy_table 过滤目标，逐项调用 pick_and_place。	tidy_table filters candidates and invokes pick_and_place.
7 Tools	Approach、Grasp、Transport、Place、Reset。	Atomic approach, grasp, transport, place, and reset operations.
8 Hardware	Step5 → RobotArm UDP → CuRobo / RT Control。	Step5 → RobotArm UDP → CuRobo / RT Control.

中文

执行时解析物体：Plan 使用 {category, color} 语义引用。感知后，Executor 才解析为 object_2 等临时 ID，因此 Plan 不绑定某一检测帧。

整理桌面展开：Planner 只提交一个 tidy_table Skill。Skill 感知场景、过滤缺失物体，再对碗、面包和杯子逐项调用 pick_and_place。

错误流：ToolResult → Skill → Executor → Server Task → 网页 Timeline，可指出物体、目标、动作、手臂、位置与底层反馈。

ENGLISH

Execution-time object resolution: Plans carry {category, color}; after perception, the Executor resolves transient object IDs, keeping Plans frame-independent.

Tidying expansion: the Planner emits one tidy_table Skill. The Skill perceives, filters unavailable objects, and invokes pick_and_place for bowl, bread, and cup.

Error flow: ToolResult → Skill → Executor → Server Task → Web Timeline, preserving object, target, action, arm, pose, and low-level feedback.

> Plan 不是 Task；Task 也不是直接的原子动作。每一层都进一步收窄权限并增加验证。
>
> A Plan is not a Task, and a Task is not a raw atomic command. Each layer narrows authority and adds validation.

ARCHITECTURE COMPARISON

VLM + Functions 与 Agent + Skill 对比

维度 / Dimension	传统 VLM + Functions	当前 Agent + Skill	价值 / Impact
模型输出	直接选择 approach / grasp / place 等函数。	只提交高层 Skill Plan。	意图与确定性运动分离。 Intent is separated from deterministic motion.
执行触发	推理后可直接 dispatch。	Plan → 审查 → 批准 → Task。	Plan 可拒绝、过期、审计，并且本身不运动。
硬件所有权	多个脚本或 bridge 可能进入硬件。	Python Agent 是唯一硬件所有者。	避免相机、机械臂、GPU 和控制状态冲突。
能力抽象	函数薄包装，流程散落在 prompt 或脚本。	Skill 封装策略；Tool 封装原子动作。	能力可测试、组合、版本化和授权。
物体引用	较早绑定 object_0 / object_1。	语义引用在感知后解析。	Plan 与临时场景编号解耦。
安全边界	依赖 prompt 与操作纪律。	白名单、参数、锁、预算和现场批准。	安全由代码实施，不依赖模型自觉。
可观测性	以日志为主，上下文易丢失。	Plan、Task、step、placement、operation 事件链。	能定位具体物体与失败环节。
迁移成本	直接运行 RobotRAM。	RobotRAM 被 Tool/Skill 包装后复用。	保留已验证能力，避免推倒重写。

架构意图：从“ VLM/LLM 作为遥控器 ”转向“ 模型负责意图，Agent 负责状态与执行，人类负责逐 Plan 批准 ”。

Architectural intent: move from “ the model as remote control ” to “ model for intent, Agent for state/execution, human for per-Plan approval. ”

LIFECYCLE & SUPERVISION

launch_robot_soul.sh 增加的生命周期能力

能力 / Capability	中文	English
统一装配	预检、构建 Harness Web、准备配置、共享 token，并按顺序启动硬件面、Harness、GUI 与 Supervisor。	Preflight, web build, configuration, shared token, and ordered startup.
双模式	start 直接启动真机；console 只启动网页/Harness，硬件保持 offline。	start boots hardware; console keeps hardware offline until on-site startup.
显式状态机	offline → starting → healthy / faulted → recovering。	UI availability and hardware readiness become separate states.
故障域分离	GUI/Harness 失败触发完整停机；硬件失败则锁定硬件并保留网页。	UI failure stops all; hardware failure preserves diagnostics while locking motion.
现场恢复	网页启动与恢复要求确认口令、生命周期锁和 sudo 授权。	Startup and recovery require confirmation, lifecycle locking, and sudo authorization.
统一退出	按顺序清理 Agent、Planner、Upper 与 RT，减少残留进程。	Coordinated shutdown reduces stale processes and ownership conflicts.

中文判断

launch_robot_soul.sh 不只是启动脚本。它把认知面、操作面与硬件面装配为具有健康检查、故障锁定和人工恢复能力的产品运行时。

- Tier 1：Harness 或统一网页连续失败，执行完整安全停机。
- Tier 2：硬件面连续失败，停止并锁定硬件，保留 Harness 与网页。
- Console 模式允许先启动操作台，再由现场人员一键初始化真机栈。

ENGLISH ASSESSMENT

launch_robot_soul.sh is more than a startup script. It assembles the cognitive, operator, and hardware planes into a supervised product runtime.

- Tier 1: repeated Harness or unified UI failure causes a complete safe shutdown.
- Tier 2: repeated hardware failure locks and stops hardware while preserving diagnostics.
- Console mode allows the operator surface to start before on-site hardware initialization.

STRENGTHS & CONSTRAINTS

优势与当前限制 / Strengths and Limits

优势 / STRENGTHS

- 安全权威分离：模型提议、操作员批准、Agent 执行、控制栈驱动。
- 可观测：Plan ID、Task 状态、阶段事件与详细失败上下文。
- 可测试：Planner、Skill、策略、审批、Origin 与状态机可独立验证。
- 可复用：pick_and_place 可被单任务与 tidy_table 组合。
- 渐进迁移：保留 RobotRAM Step1 – 5 与控制栈。
- 故障可恢复：硬件 fault 不必杀死网页和 Harness。

English: separated authority, observability, testability, reusable Skills, incremental migration, and isolated recovery.

限制 / LIMITS

- Planner 较窄：当前主要覆盖整理与单次抓放。
- 整理仍有配方：按感知过滤，但不是完全动态关系推导。
- 感知误检：有效 mask/位姿仍可能对应错误语义。
- 场景可能过期：移动物体后需要更新 world state 或重新感知。
- 状态主要在内存：重启会丢失短期 Plan、审批与历史。
- 策略双份维护：TS 与 Python 白名单、预算可能漂移。
- 单 Task 串行：暂无队列、优先级和调度。
- 取消不是急停：cancel_event 不能替代实体急停。

English: narrow planning, recipe-driven tidying, false positives, stale state, in-memory lifecycle, policy drift, and no queue.

直接结论：当前系统还不是“通用自主机器人 Agent”；它是一个安全、可审查、可扩展的任务运行时，Planner 与世界模型仍处于早期阶段。

Direct conclusion: this is not yet a general autonomous robot. It is a safe and extensible task runtime with an early-stage planner and world model.

ROADMAP

演进潜力 / Development Potential

短期 / SHORT TERM

1. 使用单一 Skill Schema 生成 TS/Python 校验与动作预算。
   Generate both policy layers from one Skill Schema.
2. 每次放置后更新世界模型；被移动目标强制重新感知。
   Refresh world state after each placement.
3. 完整保留 operation、arm、action、pose 等结构化事件。
   Preserve structured operation metadata.
4. 持久化 Plan、Task、审批、事件和执行结果。
   Persist Plans, Tasks, approvals, events, and results.

中期 / MID TERM

1. 为 Skill 增加前置条件、后置条件、风险等级、版本与重试。
   Add pre/postconditions, risk, versions, and retries.
2. 将整理任务升级为场景驱动的动态 Skill DAG。
   Generate scene-driven Skill DAGs.
3. 加入仿真回放、碰撞预检和 Plan Diff。
   Add simulation replay, collision preflight, and Plan Diff.
4. 建立有限 Task 队列，同时保持硬件 Executor 串行。
   Queue Tasks while preserving serial hardware execution.

长期 / LONG TERM

1. 使用受 Skill Manifest 约束的 LLM Planner。
   Use manifest-constrained LLM planning.
2. 多个只规划 Agent 并行提出方案，由单一执行 Agent 仲裁。
   Allow many planners but one execution authority.
3. 建立跨机器人 Capability Profile 与版本兼容。
   Build cross-robot capability profiles.
4. 用执行结果与视觉复核形成闭环学习和主动恢复。
   Learn from outcomes and visual verification.

下一阶段不应继续增加更多“ 模型可直接调用的函数 ”， 而应增加带契约、版本、风险和恢复策略的 Skill。

The next phase should add contract-driven, versioned, risk-rated, recoverable Skills—not more functions directly exposed to the model.

CONCLUSION & SOURCE MAP

结论与关键入口 / Conclusion and Source Map

必须保持的架构不变量

1. Python Agent 始终是唯一硬件所有者。
2. 模型只能提交语义 Plan，不能提交关节、CAN、shell 或任意原子调用。
3. Plan 与 Task 始终分离；真机运动始终经过独立现场批准。
4. 软件取消与 Supervisor 不能替代独立实体急停。

ARCHITECTURAL INVARIANTS

1. The Python Agent remains the sole hardware owner.
2. The model submits semantic Plans, never joints, CAN, shell, or arbitrary calls.
3. Plan and Task remain separate; motion always requires on-site approval.
4. Software cancellation and supervisors never replace the physical emergency stop.

路径 / Path	职责 / Responsibility
launch_robot_soul.sh	统一装配、start/console、Supervisor、硬件启动与恢复。 Runtime assembly and supervision.
robot_agent/server.py	Plan、Task、审批、硬件锁、事件与唯一硬件所有权。
robot_agent/planner.py	规则优先、可选 LLM 的自然语言到 Skill Plan。
robot_agent/executor.py	依赖校验、语义解析与逐 Skill 执行。
robot_agent/skills/	感知、抓放、整理、复位和启动等任务级能力。
robot_agent/tools/	感知与操作原子能力，适配 RobotRAM。
RobotRAM/scripts/step1…step5	现有感知、位姿估计、抓取规划和执行。

最终判断：Agent + Skill
架构最大的长期价值，是在不放弃现有机器人能力的前提下，把自然语言智能、确定性执行和物理安全变成三个可以独立演进的系统。

Final assessment: the long-term value is the ability to evolve language intelligence, deterministic execution, and physical safety independently while preserving existing robot capabilities.